



Deep Learning

Bias, Variance, Regularization

Dr. Mehran Safayani

 safayani@iut.ac.ir

 https://people.iut.ac.ir/en/safayani

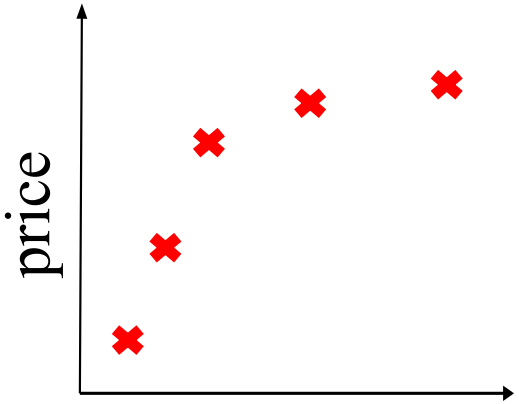
 https://www.aparat.com/mehran.safayani

 https://github.com/safayani/deep_learning_course

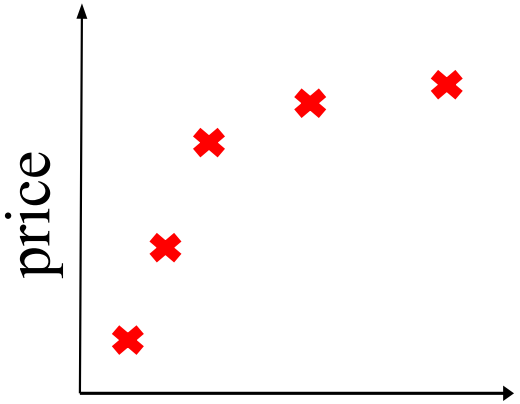


Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

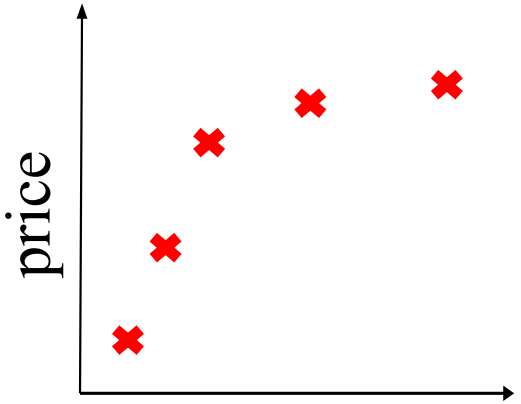
Example: Linear regression (housing prices)



$\theta_0 + \theta_1 x$



$\theta_0 + \theta_1 x + \theta_2 x^2$



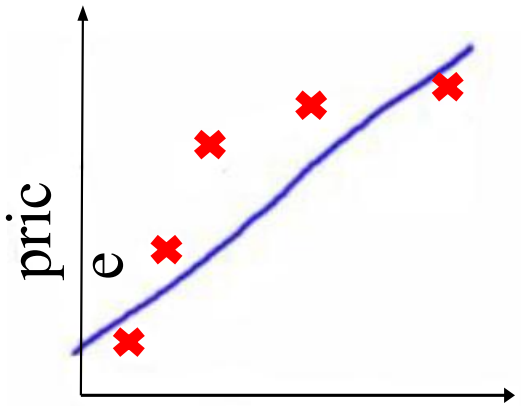
$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$



The slides are modified, based on original slides by [Andrew NG, Stanford university

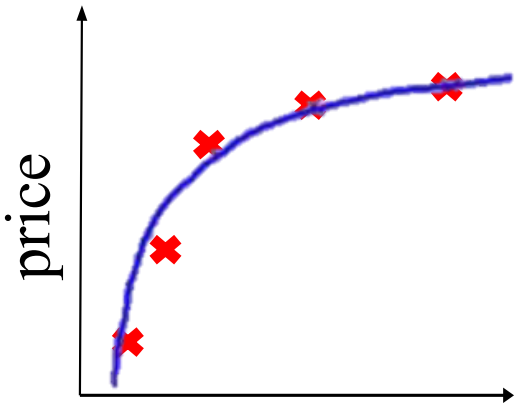


Example: Linear regression (housing prices)



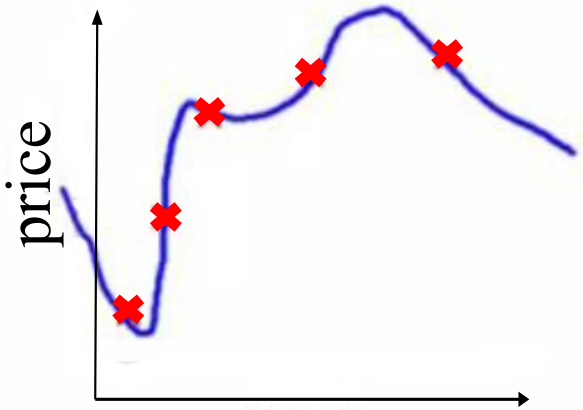
$\theta_0 + \theta_1 x$

"Underfit" **"High bias"**



$\theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

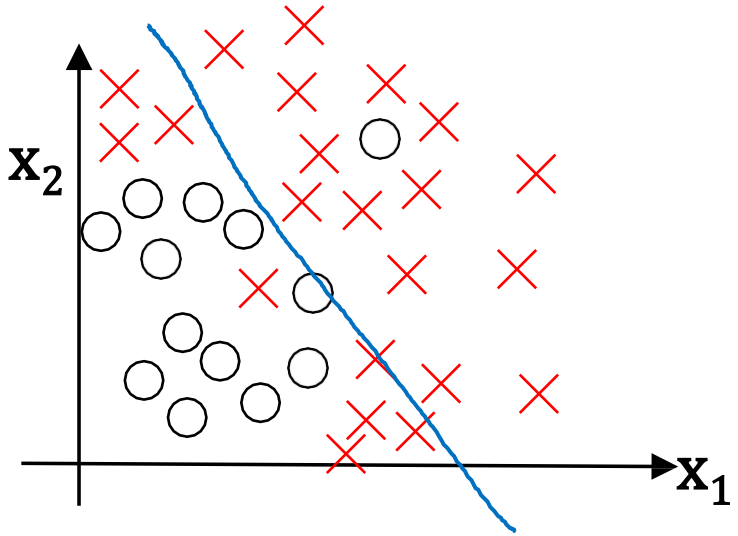


$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

"overfit" **"High variance"**

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

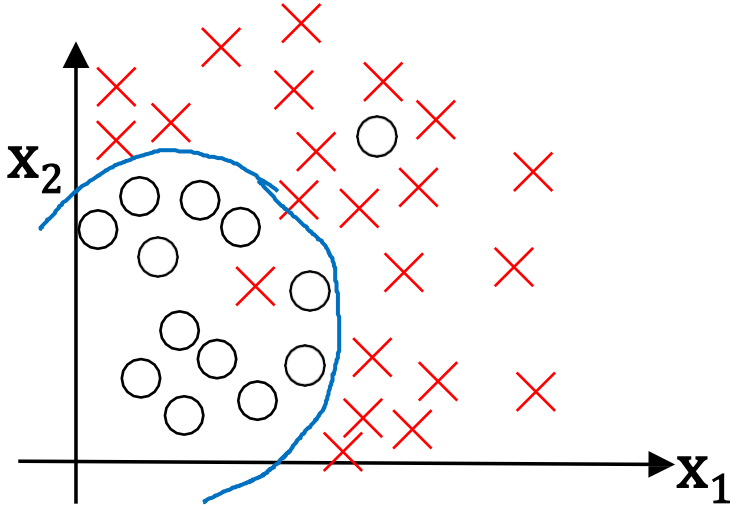
Example: Logistic regression



$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

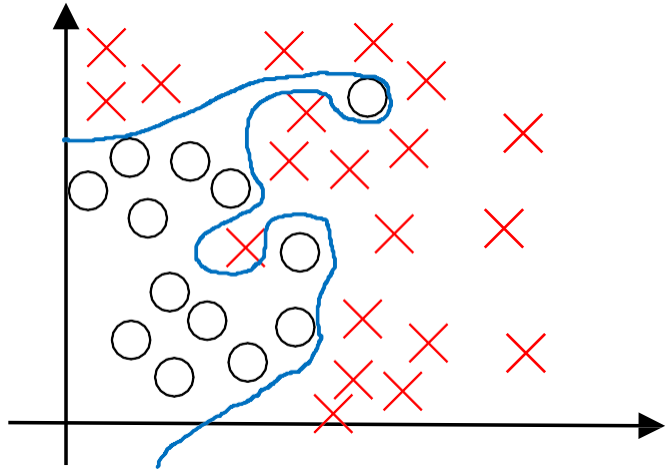
high bias

Underfitting



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$

"just right"



$g\left(\begin{matrix}\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 \\ + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \cdots\end{matrix}\right)$

high variance

Overfitting





Addressing overfitting:

x_1 = size of house

x_2 = no. of bedrooms

x_3 = no. of floors

x_4 = age of house

x_5 = average income in neighborhood

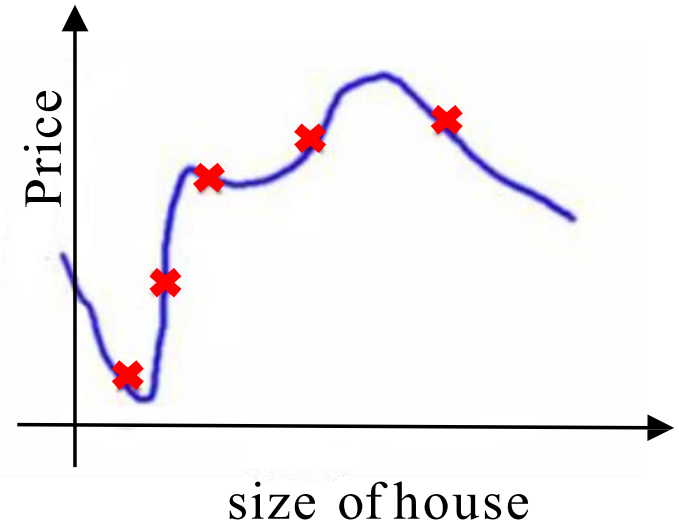
x_6 = kitchen size

.

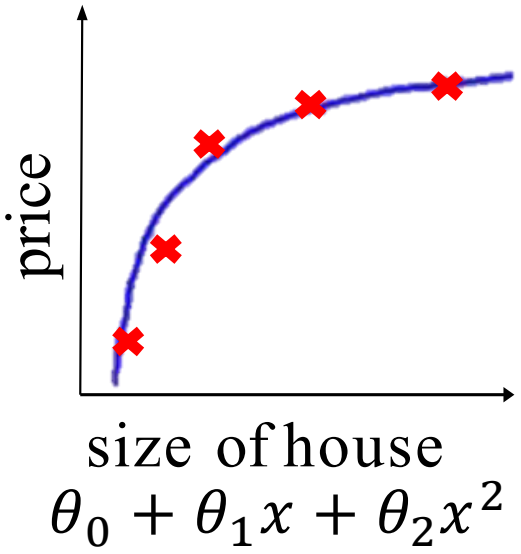
.

.

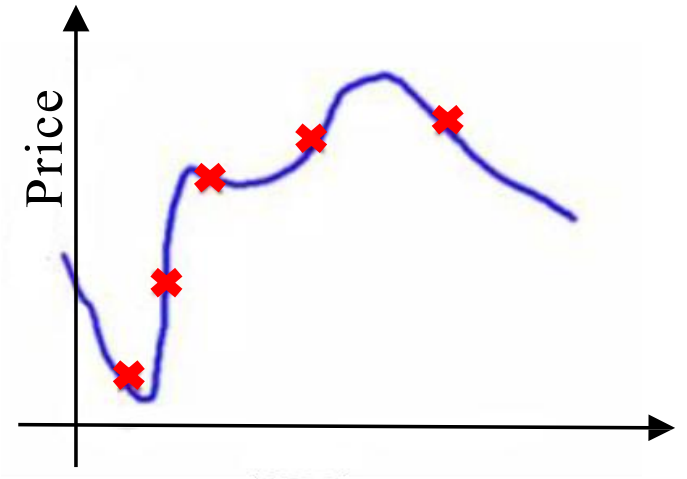
x_{100}



Intuition



$\theta_0 + \theta_1 x + \theta_2 x^2$



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

- Suppose we penalize and make θ_3, θ_4 really small.

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \underbrace{1000\theta_3^2 + 1000\theta_4^2}_{\theta_3 \approx 0 \quad \theta_4 \approx 0}$$

Regularization

- Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$
  - "Simpler" hypothesis
  - Less prone to overfitting
- Housing:
  - Features: $x_1, x_2, \dots, x_{100}$
  - Parameters: $\theta_0, \theta_1, \theta_2, \dots, \theta_{100}$

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2\right]$$

$\boldsymbol{\theta_0, \theta_1, \theta_2, \dots, \theta_{100}}$

λ = regularization parameter

lambda lambd

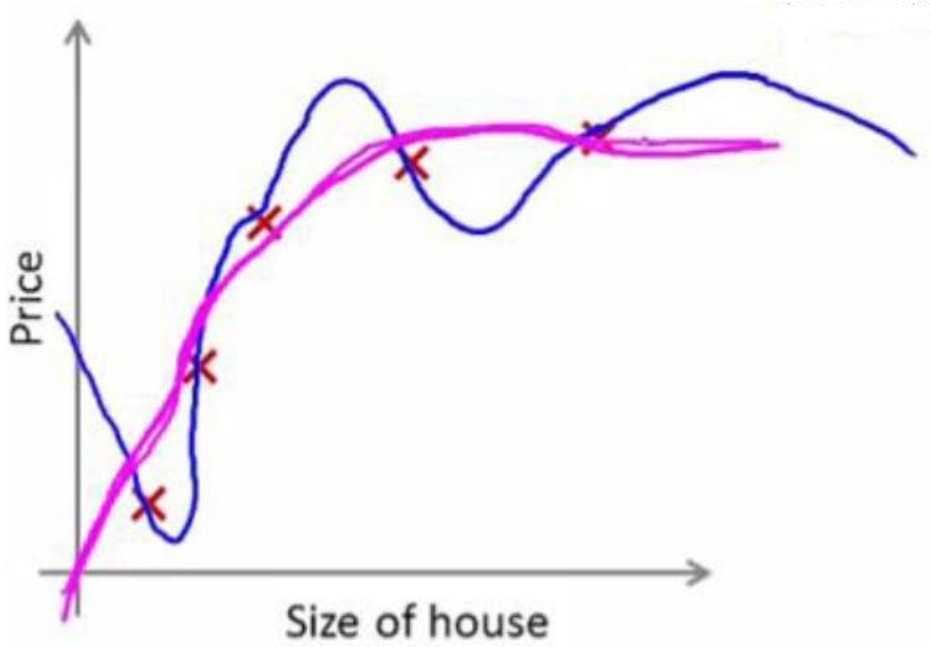




Regularization

- $J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$

$\min_{\theta} J(\theta)$

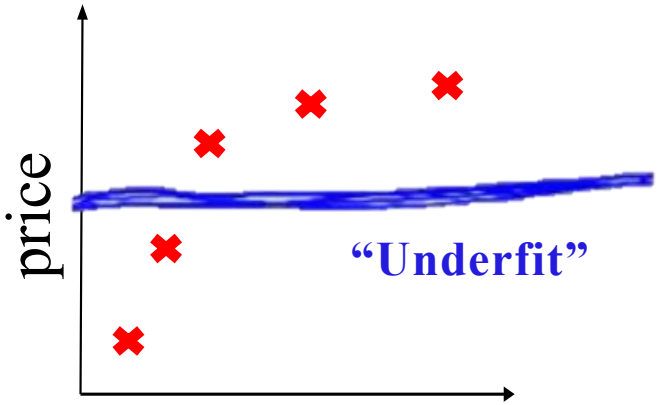


Regularization

- In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

- What if λ is set to an extremely large value (perhaps for too large for our problem, say / = 1010)?



$h_\theta(x)$

$$\theta_0 + \cancel{\theta_1 x} + \cancel{\theta_2 x^2} + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

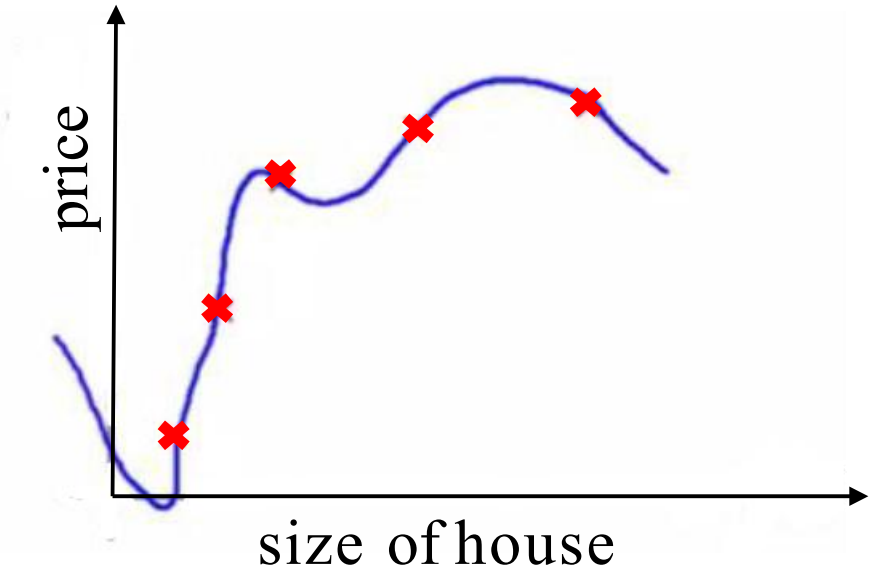
$$\theta_1, \theta_2, \theta_3, \theta_4$$

$$\theta_1 \approx 0, \theta_2 \approx 0$$

$$\theta_3 \approx 0, \theta_4 \approx 0$$

$$\boxed{h_\theta(x) = \theta_0}$$

Evaluating your hypothesis



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

- Fails to generalize to new examples not in training set.

  x_1 = size of house

  x_2 = no. of bedrooms

  x_3 = no. of floors

  x_4 = age of house

  x_5 = average income in neighborhood

  x_6 = kitchen size

  .

  .

  .

  x_{100}

Evaluating your hypothesis

- Dataset:

Size	Price	
2104	400	60% Training set
1600	330	
2400	369	
1416	232	
3000	540	
1985	300	
1534	315	20% Cross validation set (cv)
1427	199	
1380	212	20% Test set
1494	243	

$(x^{(1)}, y^{(1)})$
$(x^{(2)}, y^{(2)})$
.
.
.
$(x^{(m)}, y^{(m)})$

$(x_{cv}{}^{(1)}, y_{cv}{}^{(1)})$
$(x_{cv}{}^{(2)}, y_{cv}{}^{(2)})$
.
.
.
$(x_{cv}{}^{(m_{cv})}, y_{cv}{}^{(m_{cv})})$

$(x_{test}{}^{(1)}, y_{test}{}^{(1)})$
$(x_{test}{}^{(2)}, y_{test}{}^{(2)})$
.
.
.
$(x_{test}{}^{(m_{test})}, y_{test}{}^{(m_{test})})$





Train, Validation, & Test — Roles & Ratios

- **The Core Roles (The Exam Analogy):**
  - **Training Set (Textbook):** The data the model learns and fits parameters from.
  - **Validation Set (Practice Exam):** Used by **you** during development to tune hyperparameters, select the best model, and avoid overfitting.
  - **Test Set (Final Exam):** Locked away until the very end. Used **once** to give an honest, unbiased score of real-world performance.
- **Key Difference: Validation vs. Test:**
  - **Validation is NOT truly unseen:** Because you make decisions based on it, the model indirectly "overfits" to the validation set.
  - **Test is strictly unseen:** No decisions or adjustments are made after evaluating on test data.
- **Splitting Ratios & Sample Size:**
  - Common split: **70% / 15% / 15%** or **80% / 10% / 10%**.
  - **Golden Rule:** Absolute sample size matters more than fixed percentages. The test set must have enough samples to accurately represent the true data distribution and rare cases.





The Trap of Random Splitting (Data Leakage)

- **Why Random isn't Always Enough:**
- Random shuffling assumes every sample is independent (i.i.d). This is often false in real-world data (e.g., video, medical, sensor data).
- **Case Study: Video Frames**
- **The Problem:** If you split random frames of a car, a frame of the "same" car goes to both training and testing. The model memorizes the car rather than learning to recognize it. This leads to fake, high accuracy.
- **The Solution:**
- **Group/Day-based Splitting:** Split by distinct time periods (e.g., Train on Day 1, Test on Day 2). Ensure the model never sees "related" data during testing.





Why Test Results Can Be Misleading

- **Issue: Test Score > Training Score**
- This shouldn’t happen! It usually means:
- **Tiny Test Set:** The test set is too small and happens to contain “easy” samples.
- **Selection Bias:** The test set does not represent the real distribution.
- **How to Fix It:**
- **Grow the Test Set:** Get more data for final evaluation.
- **K-Fold Cross-Validation:** Split data into K parts. Train/Test K times, rotating the parts. Average the results for a stable score.
- **Repeated Experiments:** Run tests multiple times with different random seeds to ensure consistency





Train/validation/test error

- Training error:

$$J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

- Cross Validation error:

$$J_{cv}(\theta) = \frac{1}{2mcv}\sum_{i=1}^{m_{cv}}\left(h_\theta\left(x_{cv}{}^{(i)}\right) - y_{cv}{}^{(i)}\right)^2$$

- Test error:

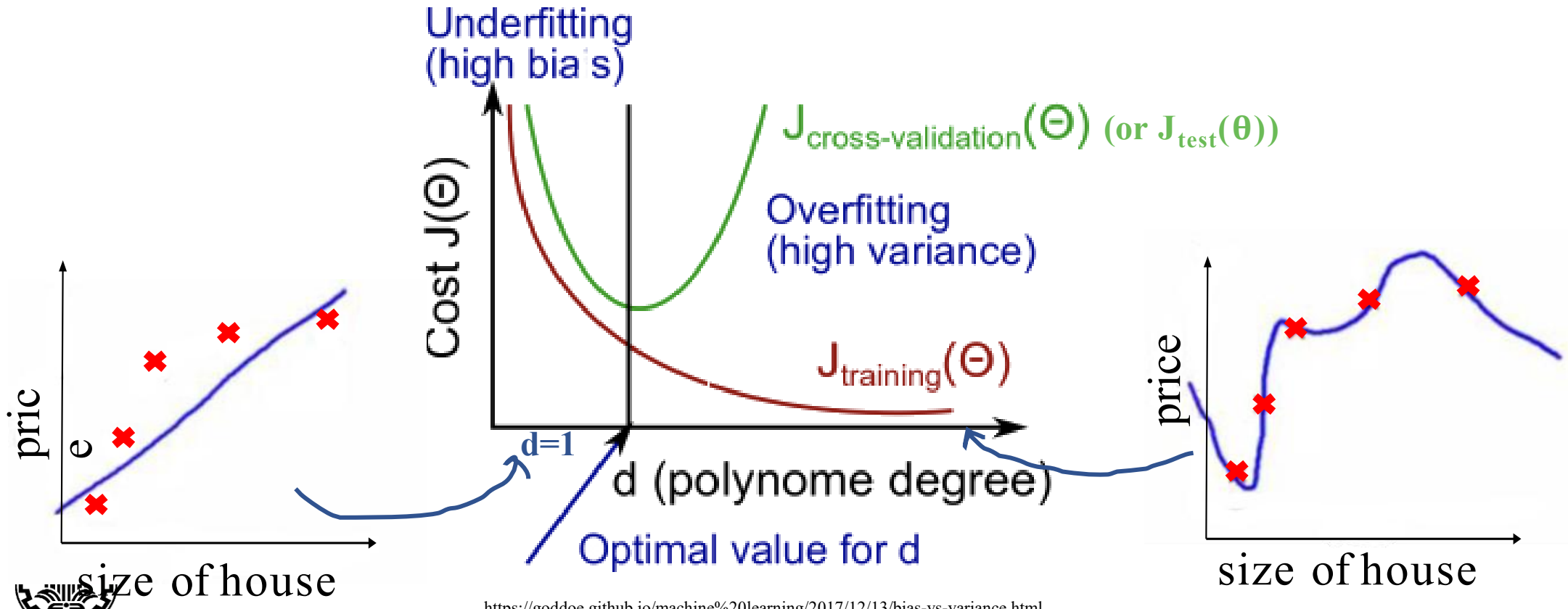
$$J_{test}(\theta) = \frac{1}{2mtest}\sum_{i=1}^{m_{test}}\left(h_\theta\left(x_{test}{}^{(i)}\right) - y_{test}{}^{(i)}\right)^2$$





Bias/variance

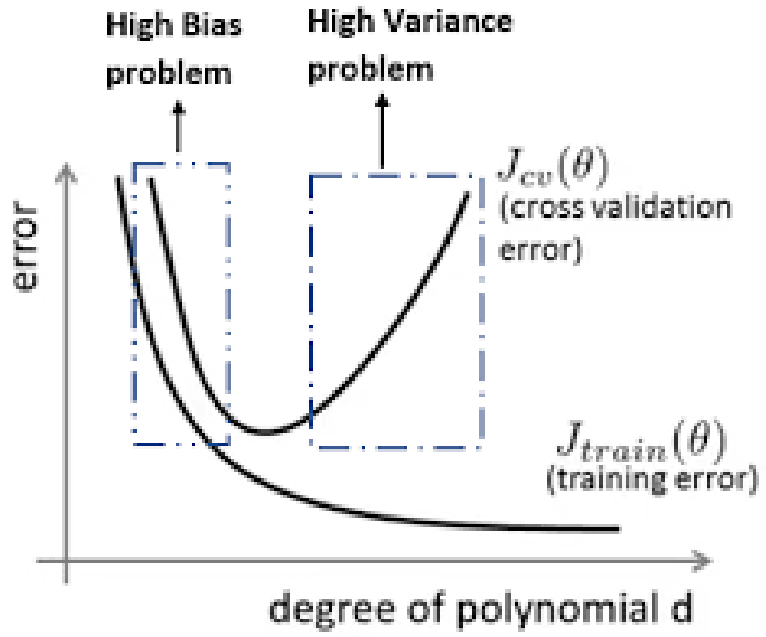
- Training error: $J_{train}(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$
- Cross validation error: $J_{cv}(\theta) = \frac{1}{2mcv}\sum_{i=1}^{m_{cv}}\left(h_\theta\left(x_{cv}^{(i)}\right) - y_{cv}^{(i)}\right)^2$ **(or $J_{test}(\theta)$)**



https://goddoe.github.io/machine%20learning/2017/12/13/bias-vs-variance.html

Diagnosing bias vs. variance

- Suppose your learning algorithm is performing less well than you were hoping. ($J_{cv}(\theta)$ or $J_{test}(\theta)$) is high.) Is it a bias problem or a variance problem?



https://towardsdatascience.com/

Bias (underfit):

$J_{train}(\theta)$ *will be high*

$J_{cv}(\theta) \approx J_{train}(\theta)$

Variance (overfit):

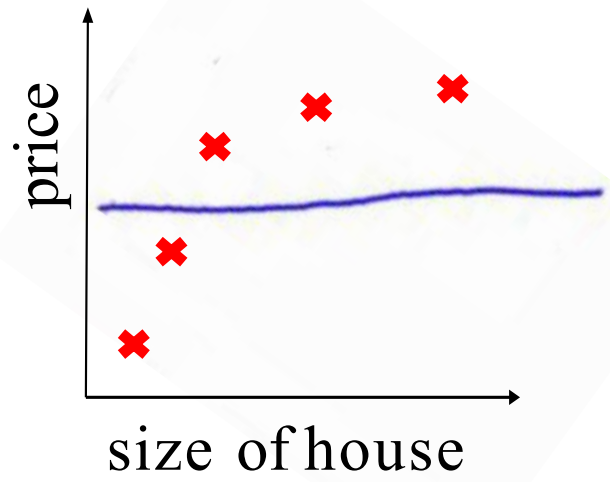
$J_{train}(\theta)$ will be low

$J_{cv}(\theta) \gg J_{train}(\theta)$

Linear regression with regularization

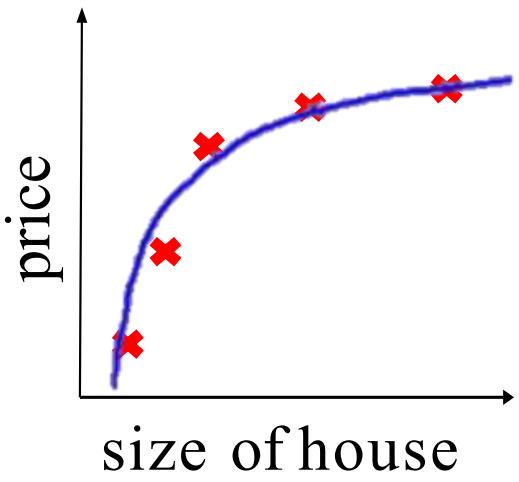
Model: $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$



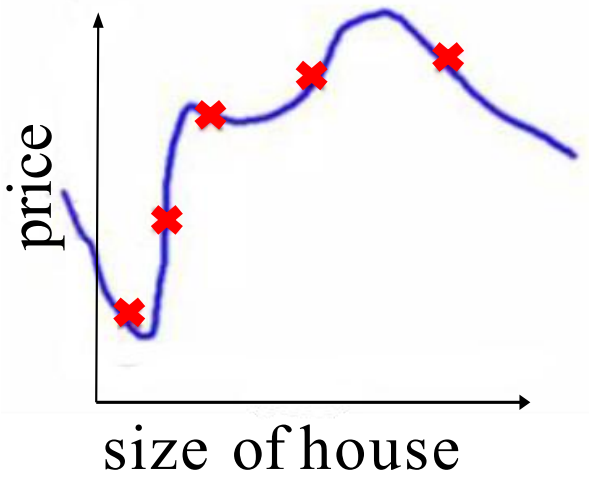
Large λ

High bias(underfit)



Intermediate λ

"Just right"



Small λ

High variance (overfit)

$\lambda = 10000.\ \theta_1 \approx 0, \theta_2 \approx 0, \ldots$

$h_\theta(x) \approx \theta_0$

$\lambda = 0$

Choosing the regularization parameter λ

Model: $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

1. Try $\lambda = 0 \rightarrow \min J(\theta) \rightarrow \theta^{(1)} \rightarrow J_{cv}(\theta^{(1)})$
2. Try $\lambda = 0.01 \rightarrow \min J(\theta) \rightarrow \theta^{(2)} \rightarrow J_{cv}(\theta^{(2)})$
3. Try $\lambda = 0.02 \rightarrow \min J(\theta) \rightarrow \theta^{(3)} \rightarrow J_{cv}(\theta^{(3)})$
4. Try $\lambda = 0.04$
5. Try $\lambda = 0.08 \rightarrow \min J(\theta) \rightarrow \theta^{(5)} \rightarrow J_{cv}(\theta^{(5)})$

.

.

.

12. Try $\lambda = 10 \rightarrow \min J(\theta) \rightarrow \theta^{(12)} \rightarrow J_{cv}(\theta^{(12)})$

10.24

Pick (say) $\theta^{(5)}$. Test error: $J_{test}(\theta^{(5)})$





Bias/variance as a function of the regularization parameter λ

- **Training error:**

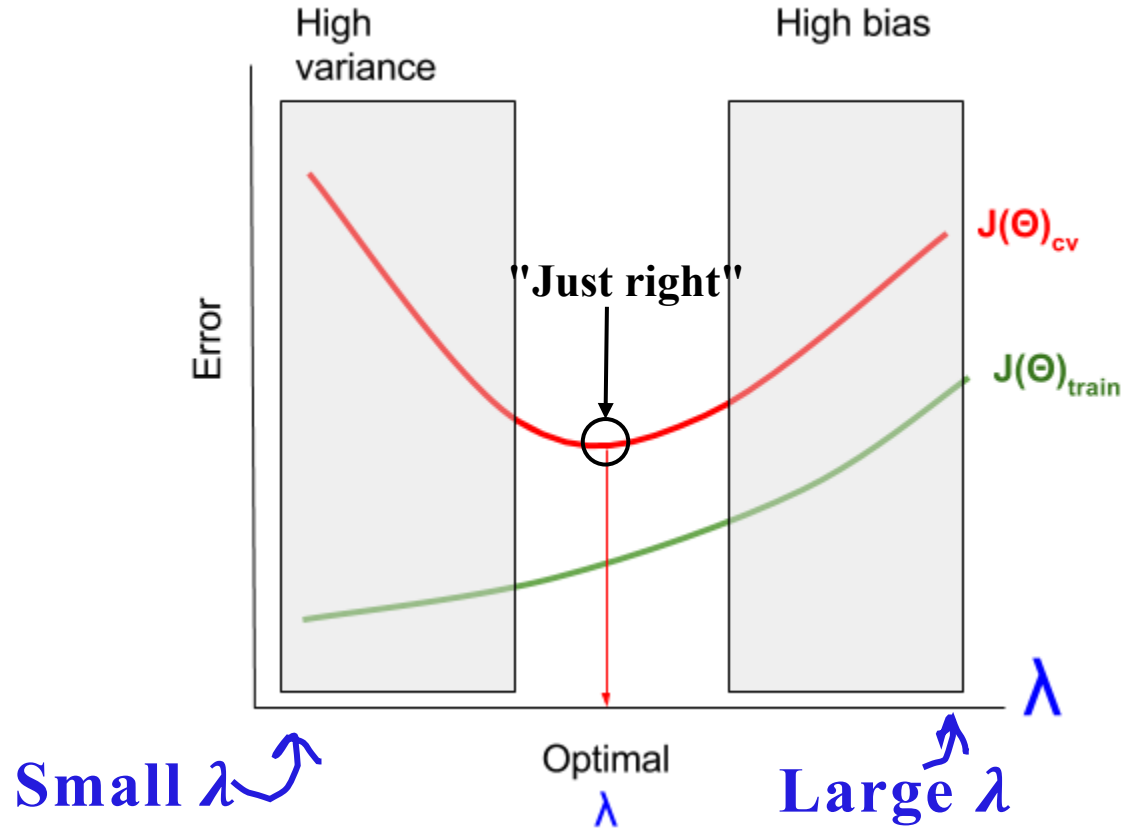
$$J_{train}(\theta)=\frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)^2+\lambda\sum_{j=1}^{n}\theta_j^2$$

- **Cross Validation error:**

$$J_{cv}(\theta)=\frac{1}{2mcv}\sum_{i=1}^{m_{cv}}\left(h_\theta\left(x_{cv}{}^{(i)}\right)-y_{cv}{}^{(i)}\right)^2$$

- **Test error:**

$$J_{test}(\theta)=\frac{1}{2mtest}\sum_{i=1}^{m_{test}}\left(h_\theta\left(x_{test}{}^{(i)}\right)-y_{test}{}^{(i)}\right)^2$$



https://stats.stackexchange.com/questions/222493/why-do-we-use-the-unregularized-cost-to-plot-a-learning-curve

Regularization

Bias/variance

- High bias → Bigger network
- High variance → Regularization

More data





Data augmentation







4



Online data augmentation (Augmentation on the fly)

- The original dataset remains relatively small on the disk.
- During the training phase, as the model requests a batch of data (for example, a batch of 32 images), the training pipeline applies random transformations to those images instantly in the computer's RAM (or GPU memory).
- The augmented version is fed directly into the model for training and is then discarded.





```
FUNCTION augment_image(image):
    // Randomly decide how many augmentations to apply (e.g., 1 to 3)
    num_ops = RANDOM_INTEGER(1, 3)

    FOR i FROM 1 TO num_ops:
        operation = RANDOM_CHOICE(["flip", "rotate", "adjust_brightness", "crop"])

        IF operation == "flip":
            // Flip horizontally with 50% chance
            IF RANDOM() > 0.5:
                image = HORIZONTAL_FLIP(image)

        ELSE IF operation == "rotate":
            // Rotate by a random angle between -15 and 15 degrees
            angle = RANDOM_FLOAT(-15, 15)
            image = ROTATE(image, angle)

        ELSE IF operation == "adjust_brightness":
            // Change brightness by a random factor
            factor = RANDOM_FLOAT(0.8, 1.2)
            image = ADJUST_BRIGHTNESS(image, factor)

        ELSE IF operation == "crop":
            // Crop a random 80% of the image and resize back
            image = RANDOM_CROP_AND_RESIZE(image, scale=0.8)

        END IF
    END FOR

    RETURN image
```





Regularization

- Logistic regression

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) + \frac{\lambda}{2m}\|w\|_2^2$$

$$L2: \|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w$$

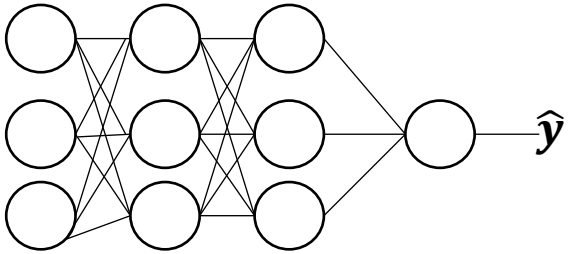
$$L_1: \|w\|_1 = \sum_{j=1}^{n_x} |w_j|$$



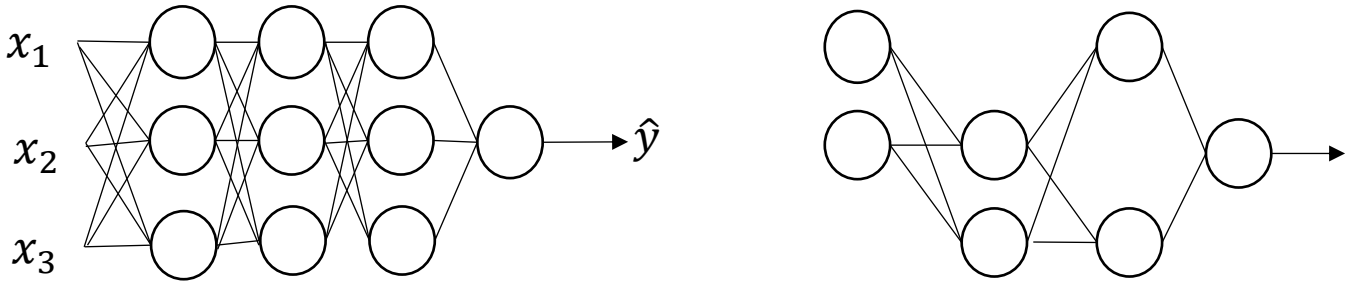


Neural Network

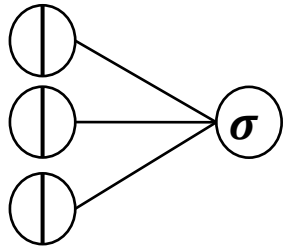
- $J\left(w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}, \dots, w^{[L]}, b^{[L]}\right) =$
- $J(w, x) = \frac{1}{m} \sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) + \underbrace{\frac{\lambda}{2m} \sum_{l=1}^{L} \left\| w^{[l]} \right\|^{2}_{F}}_{Frobenius\ norm}$
- $dw^{[l]} = (from\ backprop) + \frac{\lambda}{m} w^{[l]}$
- $w^{[l]} = w^{[l]} - \alpha\, dw^{[l]}$



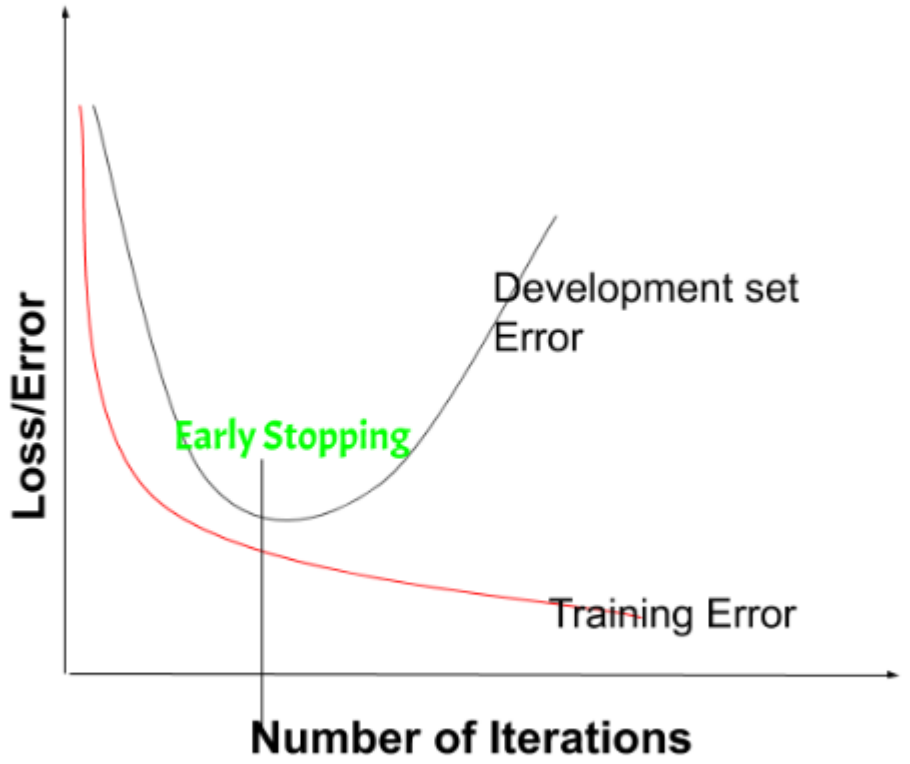
Dropout Regularization



- $d_3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep_prob = 0.8$
- $a_3 = np.multiply(a_3, d_3)$
- $a_3 /= keep_prob$



Early stopping



https://www.geeksforgeeks.org/regularization-by-early-stopping/

Old way of splitting data

- Deep learning



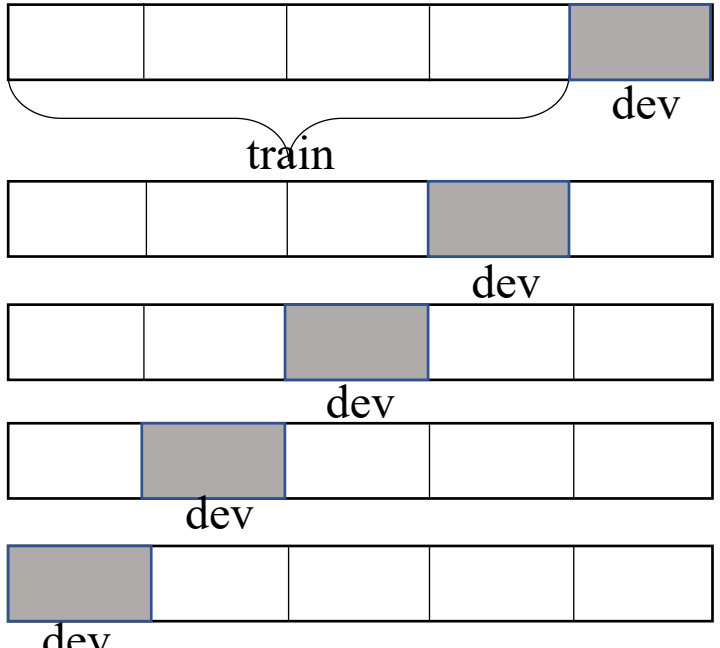
60% 20% 20%

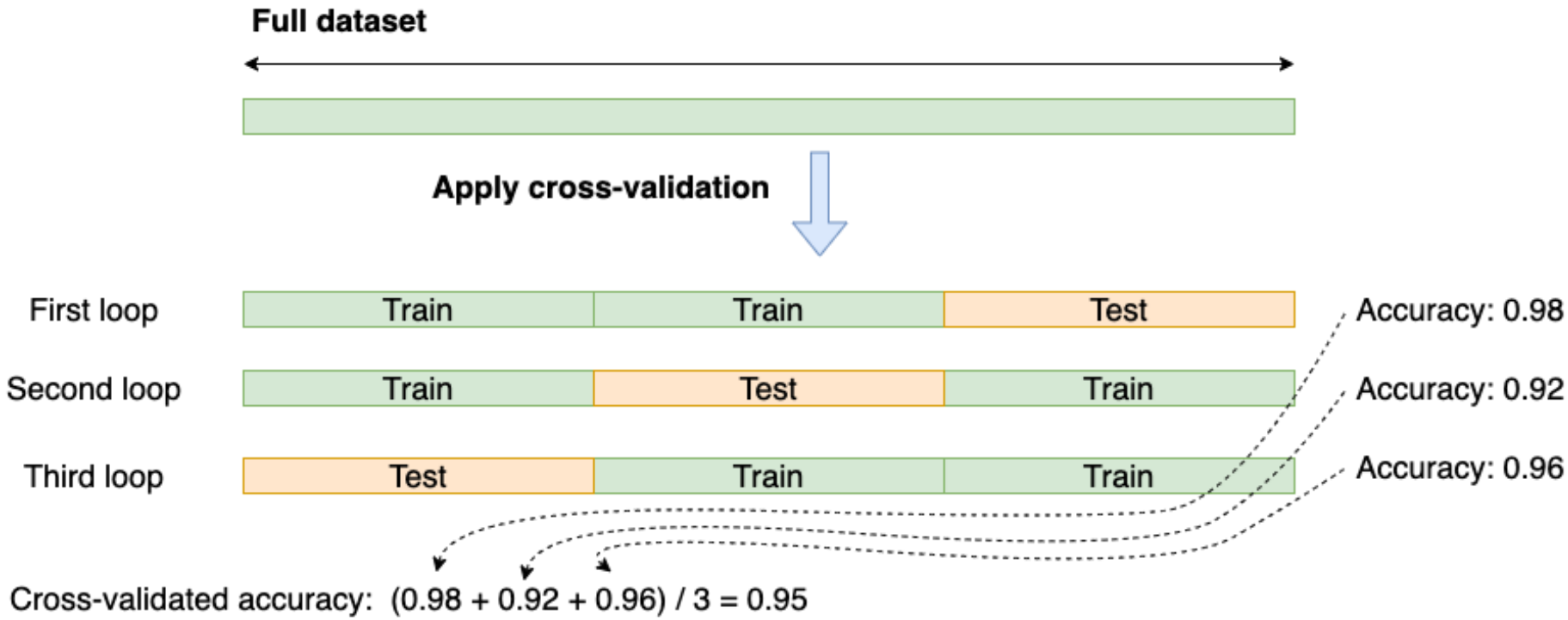
1000,000 data 1% 10,000

98% 1% 1%

99.5% 0.4% 0.1%

- K-fold cv




Full dataset
Apply cross-validation
First loop
Train
Train
Test
Accuracy: 0.98
Second loop
Train
Test
Train
Accuracy: 0.92
Third loop
Test
Train
Train
Accuracy: 0.96
Cross-validated accuracy: (0.98 + 0.92 + 0.96) / 3 = 0.95